Formal languages and compilers

Indice

Basics

Operations on strings

Review basics of strings and operations on them (API or Bioinformatics).

Operations on languages

Operation on strings can be extended to whole languages: *the operation between strings will be applied to each string of the language*.

Prefix-free language: a language where there is not any string that is prefix of another string of the language: $prefix(L) \cap L = \varnothing$. Note: ϵ is prefix to every string, including itself.

$$prefix(L) = \{y | x = yz \wedge x \in L \wedge y, z \neq \epsilon\}$$

Concatenation is defined as such: $L'L'' = \{xy | x \in L' \wedge y \in L''\}$. The powers are defined based on this definition. Some consequences:

1. $L^0 = \{\epsilon\}$
2. $L.\varnothing = \varnothing.L = \varnothing$
3. $L.L^0 = L^0.L = L$

All set operations still apply. The universal languages is defined as follows: $L_{universal} = \bigcup_n^\infty \Sigma^n$ or $L_{universal} = \neg\varnothing$.

The quotient operator is defined as such: $L'/L'' = \{y | (x = yz \in L') \wedge z \in L''\}$.

Star

To create a language with infinitely many strings we can use the star operator (concatenation closure of Kleene star):

$$L^\star = \bigcup_{h=0..\infty} L^h$$

Every string in the star closure L can be factored into substrings, each of which belongs to the language L. The star of a language can be identical to the base language.

The universal language can be written as $\Sigma^\star$. Therefore for every language over an alphabet Σ we can say that $L \subseteq \Sigma^\star$.

The star operator has the following properties:

1. **Monotonicity**: $L \subseteq L^\star$
2. **Closure w.r.t. concatenation**
3. **Idempotency**
4. **Commutes with mirroring**: $(L^\star)^R = (L^R)^\star$
5. $\varnothing^\star = \{\epsilon\}$, $\{\epsilon\}^\star = \{\epsilon\}$

The cross operator (L^+) is similar to the star, however it is not reflexive and does not contain the null power.

Regular languages

Regular languages are the simplest formal languages. A regular language can be generated by a **regular expression**, a string r defined over the letters of the alphabet that can contain a few metasymbols that follow this rules:

1. $r = \varnothing$
2. $r = a, a \in \Sigma$
3. $r = (s|t)$ (union)
4. $r = (st)$ (concatenation)
5. $r = (s)^\star$ (star)

The precedence is: star, concat, union. In the regexp ϵ is allowed.

The meaning of a regexp is to be interpreted by following this table:

Regexp	Langauge Operation
ϵ	$\{\epsilon\}$
$a \in \Sigma$	$\{a\}$
$s\|t$	$L_s \cup L_t$
st	$L_s.L_t$
$s^\star$	$L_s^\star$

Every finite language is a regular language since it can be described as the union of finitely many strings. Therefore we have that $FIN \subset REG$.

The union and iteration operator used in regexp correspond to infinitely many alternatives. By choosing a specific alternative, we can define a new regexp that defines a smaller language that the original one. This means that **given e_1, we can always derive e_2 by replacing a subexpression of e_1 with one of the possible choices**. We can then define the derivation relation.

Derivation: $e' \implies e''$ if the two regexp can be factored as $e' = \alpha\beta\gamma$ and $e'' = \alpha\delta\gamma$ with β, δ subexpressions of respectively e' and e'' and δ a choice in β.

Derivation can be applied in sequence: $e_0 \overset{n}{\implies} e_n$. We can also apply our two closures: $e_0 \overset{\star}{\implies} e_n$ and $e_0 \overset{+}{\implies} e_n$.

Using derivation, we can formally define **the language L generate by a regexp**: $L(r) = \{x \in \Sigma^\star | r \overset{\star}{\implies} x\}$

Two regexp are equivalent if they generate the same language. The language defined by a derived regexp is contained in the language defined by the deriving regexp.

There may exist several derivations that generate the same string, which however are substantially equivalent and vary only in the order of choices. **A regexp f is ambiguous if the corresponding marked regexp $f_\#$ generates two marked strings x and y such that, if the indices were removed, the unmarked strings would be identical**.

> Example: $(aa|ba)^\star a|b(aa|ba)^\star$ is ambiguous. As a matter of fact, the marking $(a_1a_2|b_3a_4)^\star a_5|b_6(a_7a_8|b_9a_{10})^\star$ generates the two strings $b_3a_4a_5$ and $b_6a_7a_8$ that project ambiguously onto the same phrase baa.

Other derived operations we can define on regexp are: repetition, optionality and interval of an ordered set.

If we **allow usage of set operation in regexp (intersection, complement and difference) we obtain the extended regular expressions**.

Regular languages are **closed** to: **concatenation, union and star**. This means that regular languages are also **closed with regards to the derived operators** i.e. cross, repetition, optionality, etc. Regular languages are also closed w.r.t the set operations (intersection, complement and difference) and mirroring.

Another property is that **REG is the smallest set such that**:

1. REG **contains all finite languages**
2. REG is **closed w.r.t concatenation union and star**

Later we will define the LIB set (context-free languages) which satisfies the above properties, but is not the smallest since $REG \subset LIB$ holds. Moreover LIB is closed also w.r.t the set operations and mirroring.

Linguistic abstractions

Linguistic abstractions **transform the phrases of a real language and gives them a simpler form, called abstract representation**. To do so, the **symbols of the effective alphabet are discarded and replaced by those of the abstract alphabet**.

At the abstract level, the typical structures of most artificial languages can be obtained by the composition of few elementary paradigms, and by the basic language operations such as concatenation, union and iteration.

Substitution (replacement) is the operation that replaces terminal characters of the source language with the phrases of the target or destination language.

Compiler design makes reference to the abstract language to process, rather than to the effective language.

Lists

One structure very important in artificial languages, modelled by regexp, are lists. A list is a sequence of a number of elements, not fixed in advance; **it can be generated by e^+ (or $e^\star$ if empty lists are admitted)**. The element e **can be a terminal symbol or a compound object** of some kind (for instance a list).

Regexp for a list with separators ad start/end markers: $ie(se)^\star f$ **(or alternatively** $i[e(se)^\star]f$.

Context free grammars

A grammar is **defined by 4 entities**:

1. V is the **non-terminal alphabet**, a set of non-terminal symbols
2. Σ is **the terminal alphabet**, a set of characters that constitute the phrases
3. P a set of **syntactic rules**
4. $S \in V$ a **particular non-terminal called axiom**

To avoid confusion, metasymbols like $\rightarrow, |, \cup, \epsilon$ must not appear among terminal and non terminal symbols. **Moreover the terminal and non-terminal alphabets bust be disjointed**.

Grammars have been already discussed during **API**.

In order to generate infinitely many strings, it is necessary and sufficient to have recursive grammars. A grammar is recursion free if and only if the graph of the binary relation produced is acyclic.

Reduced form

A grammar should be **clean: every non-terminal is defined and that each of them actually contributes to generating the string**. A grammar is in **reduced form if**:

1. **Every non terminal is reachable by the axiom**, i.e there exists a derivation as follows: $S \overset{\star}{\Longrightarrow} \alpha A \beta$
2. **Every non terminal A generates a non empty set of strings** $L_A(G) \neq \emptyset$

   A grammar can be **reduced by eliminating all undefined or unreachable non-terminal symbols**. An **algorithm** exists for cleaning up grammars:

3. We construct the set DEF of defined non-terms. First we define $DEF := \{A | (A \to u) \in P, u \in \Sigma^\star\}$. The we apply the following transformation until it converges: $DEF := DEF \cup \{B | (B \to D_1 D_2 \dots D_n) \in P\}$
4. The identification of reachable non-terminals can be reduced to finding a path in the graph associated with the binary relation *produce*:

$$A \overset{produce}{\to} B \iff$$
$$A \to \alpha B \beta \quad A \neq B \quad \alpha, \beta \text{ strings}$$

A third property is often required for a grammar to be in reduced form: **the grammar may not have circular derivations**.

Syntax trees, equivalence and ambiguity

A **syntax tree is a directed acyclic graph such that for every pair of nodes there exists one and only one path that connects them**. It represents graphically the derivation process. It assigns a *root-node-leaf* relation. The sequence of leaves, from left to right, are called frontier. The degree of a node is the length of the rule. The root of the tree is the axiom, while the frontier is the final generated phrase.

Weak equivalence: two grammars G and G' are weakly equivalent if and only if they generate the same language.

Strong equivalence (or structural equivalence): two grammars G and G' are strongly equivalent if and only if they generate the same language and if G and G' assign to every phrase syntactic trees that can be considered structurally equivalent.

Strong equivalence implies weak equivalence. Strong equivalence is decidable, while weak is undecidable.

The basic regular operations (union, concat and star) applied to free languages still yield free languages.

Syntactic ambiguity: a phrase x of the language defined by G is said to be ambiguous if it admits two or more different syntactic trees. A grammar G is said to be ambiguous if it generates an ambiguous string.

The **ambiguity degree of a phrase x in the language $L(G)$ is the number of different syntax trees that X admits**. The ambiguity degree can be infinite. The **ambiguity degree of a grammar is the maximum ambiguity degree of the strings generated by the grammar**.

The problem of finding if a grammar is ambiguous or not is undecidable. Since ambiguity is difficult to remove *a-posteriori*, it is advisable to remove at construction (*a-fortiori*).

1. **Two-sided recursion is always ambiguous** since it always allows for two or more derivations.
2. If two languages $L(G_1)$ and $L(G_2)$ **have a non empty intersection, then the union grammar is surely ambiguous**.
3. **Concatenating** two languages may give rise to **ambiguity if there exists a suffix of a phrase of the former language that is a prefix of a phrase of the latter language**.
4. **Regex can be ambiguous**

Inherently ambiguous language: every grammar that generates it is necessarily ambiguous. A classical example of such a language is:

$$L = \{a^i b^j c^k | (i, j, k \geq 0)((i = j) \vee (j = k))\}$$

Normal forms

Normal form: a normal form is a form that imposes some restrictions, without reducing the family of generated languages. There are some transformations to put a grammar into an equivalent normal form.

Simple transformations

1. We can **always remove the axiom from a rhs** by introducing a new axiom S_0 such that $S_0 \to S$
2. **Non-nullable normal form**: a form such that no derivation is as follows $A \overset{+}{\Longrightarrow} \epsilon$
3. A copy rule is a rule of the type $A \to B$ type (A, B non-terminals). They can always be eliminated by substitution.

Chomsky normal form

The production are only of type:

1. **binary rule**: $A \to BC$ with B, C non-terminals
2. **terminal rule**: $A \to \alpha$ with α terminal
3. If the language contains the empty string, add $S \to \epsilon$

The syntax tree of a language in this form is strictly a binary tree.

Non-left recursive form

Left recursion can be turned into right recursion by means of simple algorithm. Non-left recursive form is a form that has no left-recursive rules.

Given:

$$A \to A\alpha_1 | \dots | A\alpha_n | \beta_1 | \dots | \beta_m$$

With:

- α_x a sequence of terminal and non-terminals that does not start with A
- β_x a sequence of terminal and non-terminals

We can replace the rule above with the two rules below:

$$A \to \beta_1 A' | \dots | \beta_m A'$$
$$A' \to \alpha_1 A' | \dots | \alpha_n A' | \epsilon$$

Real time normal form and Greibach normal form

In the **real time normal form** every rule has the form

$$A \to a\alpha \text{ where } a \in \Sigma, \alpha \in (\Sigma \cup V)^\star$$

The **Greibach normal form is a special case of the above**, where the rules need to be of this form:

$$A \to a\alpha \text{ where } a \in \Sigma, \alpha \in V^\star$$

Uni-linear grammars

A grammar is uni-liear if the productions are either:

1. $A \to uB$ (right linear)
2. $A \to Bu$ (left linear)

A uni-linear grammar always generates a regular language.

A **self-inclusive derivation** is a derivation of form $A \overset{+}{\Longrightarrow} uAv$ with u, v non-empty. **The absence of self-inclusive derivations is sufficient to say that the grammar generates a regular language**.

Extended BNF

To make free grammars more expressive, we can **allow regular expression in the rhs of the productions**. Since the family of free grammars is closed with regards to the regular operators (union, star and concatenation), **EBNF have the same generating power of free grammars**.

A non-extended grammar is ambiguous also when conceived as extended. However the presence of regexp in the rules may give rise to specific ambiguity forms.

Finite state automata

Basics about FSA have been done in API.

The syntax diagram of the automaton is the dual form of the normal state-transition graph: we transform nodes into arcs into nodes and viceversa.

An automaton may contain useless parts, which do not give any contribution process and can be eliminated:

1. A state is accessible (reachable) from another state if there is a computation that moves the automaton from one state to the other
2. A state is postacessible (defined) if some final state can be reached from it
3. A state is useful if it is both accessible and postaccessible
4. An automaton in in clean form if every state is useful

Every automaton has an equivalent clean form. To reduce an automaton we first identify all useless states, then strip the off the automaton with all their incoming and outgoing arcs.

For every finite language there exists one and only one deterministic finite state recognizer that has the smallest possible number of states, which is called the minimal automaton. To reduce an automaton to its minimal form we need to introduce undistinguishability:

- A state p is undistinguishable from a state q if an only if for every input string x either both the next states $\delta(p, x), \delta(q, x)$ are final or neither one is.
- A state p i distinguishable from q if and only if either:
  - p is final and q is not (or viceversa)
  - $\delta(p, a)$ is distinguishable from $\delta(q, a)$

Two undistinguishable states can be merged and thus the number of states of the automaton can be reduces, without changing the recognized language. Undistinguishability is a binary relation and is an equivalence relation.

It is possible to prove that the minimal deterministic automaton is unique. This cannot be done for the non-deterministic one. The uniqueness of the minimal form offers a way to check whether two deterministic finite state automata are equivalent.

Generalized automaton

Assume the that the initial and final states are unique and do not have incoming or outgoing arcs. We can construct the generalized finite automaton, an automaton where its arcs may be labeled with regular expressions, not only individual characters.

We can eliminate one by one all internal nodes (nodes that are neither the start or a terminal node) and add one arc labeled by a regex. At the end only the starting and terminal node with only one arc from the one to the other. The regex that labels such arc generates the complete language recognized by the original automaton.

From non-deterministic to deterministic

For efficiency, usually the final version of a finite automaton ought to be in deterministic form. Every non-deterministic automaton can always be transformed into an equivalent deterministic one. Consequently, every right-linear grammar has a non-ambiguous equivalent.

The algorithm is structured into two phases:

1. Elimination of spontaneous moves
2. Replacement of non-deterministic moves by changing the automaton state set

From regex to a FSA

There are a few algorithms to transform a regex into an automaton. We will present two: Thompson and Berry-Sethi.

Thompson method

We first subdivide the regex into subexpressions, until we reach the atomic constituents. Then we construct the subexpression recognizer and connects them. The result is a non-deterministic automaton with spontaneous moves.

Berry-Sethi method

It constructs a deterministic recognizer without spontaneous moves, but of size often larger that the Thompson one.

We need the following definitions (local states). Given a language L over Σ:

1. Start set, or initials; $Ini(L) = \{a \in \Sigma | a\Sigma^\star \cap L \neq \varnothing\}$
   - $Ini(\varnothing) = \varnothing$
   - $Ini(\epsilon) = \varnothing$
   - $Ini(a) = \{a\}\, \forall a$
   - $Ini(e \cup e') = Ini(e) \cup Ini(e')$
   - $Ini(e.e') = Null(e)\ ?\ Ini(e) \cup Ini(e')\ :\ Ini(e)$
   - $Ini(e^\star) = Ini(e^+) = Ini(e)$
2. End set, or finals: $Fin(L) = \{a \in \Sigma | \Sigma^\star a \cap L \neq \varnothing\}$
   - $Fin(\varnothing) = \varnothing$
   - $Fin(\epsilon) = \varnothing$
   - $Fin(a) = \{a\}\, \forall a$
   - $Fin(e \cup e') = Fin(e) \cup Fin(e')$
   - $Fin(e.e') = Null(e')\ ?\ Fin(e) \cup Fin(e')\ :\ Fin(e')$
   - $Fin(e^\star) = Fin(e^+) = Fin(e)$
3. Adjacency set, or digrams: $Dig(L) = \{x \in \Sigma^2 | \Sigma^\star x \Sigma^\star \cap L \neq \varnothing\}$
   - $Dig(\varnothing) = \varnothing$
   - $Dig(\epsilon) = \varnothing$
   - $Dig(a) = \varnothing\, \forall a$
   - $Dig(e \cup e') = Dig(e) \cup Dig(e')$
   - $Dig(e.e') = Dig(e) \cup Dig(e') \cup Fin(e).Ini(e')$
   - $Dig(e^\star) = Dig(e^+) = Dig(e) \cup Fin(e).Ini(e)$
4. Null: $Null(e) = true \iff \epsilon \in L(e)$
   - $Null(\varnothing) =$ false
   - $Null(\epsilon) =$ true
   - $Null(a) =$ false $\forall a$
   - $Null(e \cup e') = Null(e) \vee Null(e')$
   - $Null(e.e') = Null(e) \wedge Null(e')$
   - $Null(e^\star) =$ true
   - $Null(e^+) = Null(e)$

First we start by linearizing the regex e into $e_\#$: we numerate each generator such that each terminal appears only once in the expression. Let us consider these regex terminated by $\dashv$. We define the set $Fol(c_\#)$ in this way $Fol(a_i) = \{b_j | a_i b_j \in Dig(e_\# \dashv)$. Fol is just a different way of expressing Dig.

A machine executable algorithm for constructing the FSA is the following:

```
q0 = Ini(e#, -|)
Q = { q0 }
transitions = {}
while (exists q in Q such that q is unmarked) do
  mark_as_visited(q)
  for each (c in alphabet) do
    q' = merge_all(Fol(c#) where c# in alphabet# such that c# in q)
    if q.empty() then
      if !Q.contains(q') then
        unmark(q')
        Q = merge(Q, {q'})
      end
      transitions = merge(transitions, { q ->(c) q' }
    end
  end
end
```

Complement of an FSA

To **complement an FSA**, remember that REG is closed w.r.t complement we can use a simple algorithm:

1. We make sure that **complete the starting automata with an error state**.
2. We **swap final and non final states**.

For this algorithm to work **the starting automaton must be deterministic**.

Since REG is closed for complement, **we can prove that it is also closed w.r.t the other set operation** since they can be reduced to union and complement:

- $L \cap L' = \neg(\neg L \cup \neg L')$
- $L \setminus L' = L \cap \neg L'$

Cartesian product

It is a very common construction of formal languages, where **a single automaton simulates the computation of two automata that work in parallel**. It is very useful for constructing the intersection automaton as the product automaton directly recognizes the intersection automaton.

Suppose **both automata do not contain ϵ-moves**, the **state set of the product machine is the cartesian product of the state sets of the automata**. Each product state is a pair $\langle q', q'' \rangle$ where each side is a state of one of the machines. The product machine **has a move if an only if the projection of such a move onto the left/right component is a move of the first/second automaton**. The **initial and final state sets are the cartesian products of the initial and final state sets of the automata**, respectively.

If a string is accepted by the product automata, it is either accepted by both automata or rejected by at least one.

The product automaton may have useless states and may not be minimal. If either automaton is deterministic, then their product is deterministic.

Free languages and PDAs

PDAs have been discussed in API.

From grammar to PDA

Grammar rules can be viewed as the instructions of a non-deterministic PDA with only one finite state[1]**.** The initial action is the grammar axiom.

[1] see section 4.3 in API notes.

At every step the automaton **chooses non-deterministically one applicable grammar rule and executes the corresponding move**. The input string is recognized when and only when it is completely scanned and the stack is empty.

The **construction from grammar to automaton can be inverted and used to obtain the grammar by starting from the pushdown automaton, if it is one state**.

Theorem - the family of **free languages generated by free grammars coincides with the family of the languages recognized by one state push down automata** (in general non-deterministic).

Since the automaton cannot guess the correct derivation, it has to examine all possibles,. A **string is accepted by two or more different computations if and only if the grammar is ambiguous**.

We can have three different acceptance modes:

1. by final state
2. by empty stack
3. all of the above.

For the family of (non-deterministic) push down automata with states, the three acceptance modes listed above are equivalent.

Eliminating spontaneous moves

A generic PDA may execute an unlimited number of moves without reading any input character. This happens iff it enters a loop made only of ϵ-moves. **It is always possible to build an equivalent automaton with no spontaneous loops**.

On-line mode

A generic PDA operates in on-line mode if it decides whether it accepts the string as soon as it reads the last character of the input string without executing ulterior moves. It is always possible to build an equivalent automaton that works in on-line mode.

Free languages and deterministic languages

The **language accepted by a generic PDA is free**. Since every free language can be recognized by a non-deterministic one-state PDA it follows that **the family CF of context free languages coincides with that of the languages recognized by unrestricted push down automata and, more specifically, by the one state non-deterministic push down automata**.

We study more in detail the deterministic recognizers and their languages, due to their efficiency. **The family DET of deterministic free languages is strictly contained in the family CF of all the free languages**.

Syntax analysis

Given a grammar G, the syntax analyzer reads a source string and if the string belongs to the language $L(G)$ then it exhibits a derivation or builds a syntax tree of the string. If an error is found, it notifies it. **If the source is ambiguous, then the result is forest of trees or set of derivations**.

There are **two analyzer classes** depending on whether the derivation is rightmost or leftmost:

- **Bottom-up analysis**: rightmost derivation in inverse order. Constructs a tree **from leaves to root through reductions**.
- **Top-down analysis**: leftmost derivation in direct order. Construct the tree **from root to leaves through expansions**.

Network of FSA

Suppose the grammar G is in **extended form**. Each non-terminal has one rule where the rhs is a regular expression. The regexp defines a regular language, therefore **there is a finite automaton M_a that recognizes said regexp**. A **transition of M_a labeled with a non-terminal B is interpreted as a call to the automaton M_b**.

Lets establish some **definitions** and terminology:

- **Machines**: the finite state automata of the non-terminals of G

- **Automaton**: the PDA that analyzes $L(G)$
- **Network**: the set of all the machines G
- $S \to \sigma, A \to \alpha, B \to \beta, \ldots$: grammar rules
- $R_S, R_A, R_B, \ldots$: **regular languages** over the alphabets
- $M_S, M_A, M_B, \ldots$: **DFSA** recognizing $R_S, R_A, R_B, \ldots$
- $\mathscr{M} = \{M_S, M_A, M_B, \ldots\}$: **machine network**

Let us suppose that **all machine states are distinct**. Let $R(M_a, q)$ the regular language accepted by M_a starting from q, the final language (in general context free)

$$L(M_a, q) = \{y \in \Sigma^\star | \exists \eta : \eta \in R(M_a, q) \wedge \eta \overset{\star}{\Longrightarrow}_G y\}$$

We set an additional constraint to the machines of the network, so that the **initial state 0_a of a machine A is never reentered after the start of the computation**. This constraint is easily fulfilled, however this means that machines are no longer minimal.

Overview of top-down analysis

Using the framework set up in the previous section, we can implement top-down analysis as the emulation of the different DFSA: **each transition acts as a "function call" to different automatons**.

When the finite automaton has a **bifurcation state**, to decide which direction to take, **we must watch all the symbols that appear on the arcs that leave the state, included the final "darts" of the machine**.

Top-down analysis **does not work if the symbol sets on the arcs that leave the same state (guide-sets) are not disjoint**.

Bottom-up analysis

We will now explain bottom-up analysis, otherwise called $ELR(k)$ where k is the lookahead set and we will consider equal to 1.

For the analysis we need to add the **follow-sets** (often called lookahead set) for the non-terminals. The follow set **allows us to decide when to**:

1. Go on with the analysis and run an automaton transition (**shift** move)
2. Recognize a non-terminal and build a sub-tree (**reduce** move)

The **systematic way** to construct an ELR syntax analyzer is the following:

1. **Construction of the pilot graph**. The pilot drives the PDA. In **each macro-state** (m-state) the pilot **incorporates all the information about any possible phrase that reaches the m-state**. Each m-state contains machine-state with lookahead.
2. The **m-states are used to build a few analysis threads in the stack**: each **correspond to possible derivations** (computations of the machine network or patch with ϵ-arcs at each machine change, labeled with the scanned string).
3. **Verification of determinism** on the pilot graph. These conflicts can occur:
   - A **shift-reduce conflict** signifies that both a shift and a reduction are possible in a parser configuration
   - A **reduce-reduce conflict** signifies that two or more reductions are similarly possible
   - A **convergence conflict** occurs when two different parser computations that share a lookahead token lead to the same machine state.
4. If the determinism test is passed, the PDA can analyze the string deterministically

Definitions

- **Initials**: similar to Berry-Sethi, is the set of characters found starting from state q_a of machine M_A of the net $\mathscr{M}$.

  It can be defined by three cases:

  1. $a \in Ini(q_a)$ if exists $q_A \overset{a}{\to} r_A$
  2. $a \in Ini(q_a)$ if exists $q_A \overset{B}{\to} r_A$ and $a \in Ini(0_B)$

3. $a \in Ini(q_a)$ if exists $q_A \overset{B}{\rightarrow} r_A$ and $L(0_B)$ is nullable and $a \in Ini(r_A)$

- **Item**: $\langle q_B, a\rangle \in Q \times (\Sigma \cup \{\dashv\})$ where a is valid lookahead for for the current machine M_B in state q_B.

  Two or more items with the same state can be grouped into one item. An item with a machine final state is said to be a reduction item.

  When parsing start, the first item of the axiomatic machine is encoded as $\langle 0_S, \dashv\rangle$

- **Closure**: A function that computes the set of the machines that are reached from a given state q through one or more invocations, without any intervening state transition. For each reachable machine M_B, represented by the initial state 0_B, the function returns also any input character b that can legally occur when the machine terminates; such a character is part of the look-ahead set.

$$closure(C) = C$$

$$\langle 0_B, b\rangle \in closure(C) \text{ if } \begin{cases} \exists \langle q, a\rangle \in C & \text{and} \\ \exists q \overset{B}{\rightarrow} r \in \mathscr{M} & \text{and} \\ b \in Ini(L(r) \cdot a) \end{cases}$$

- **Shift**: corresponds to a transition in a machine Y
  - If $X = c$ is a terminal symbol, then shift is a PDA move that reads a char c of the input
  - If X is a non terminal symbol, then shift is a PDA ϵ-move after a reduction $z \rightarrow X$ and it does not read any input

$$\begin{cases} \vartheta(\langle p_A, \rho\rangle, X) = \langle q_A, \rho\rangle & \text{if } p_A \overset{X}{\rightarrow} q_A \text{ exists} \\ \varnothing & \text{otherwise} \end{cases}$$

- **Macro-state**: a set of items
- **The pilot**: a DFA defined by
  - The set R of m-states
  - The pilot alphabet is the union $\Sigma \cup V$ of the terminal and non-terminal alphabets (also named grammar symbols)
  - The initial m-state, I_0, is the set $I_0 = closure(\langle 0_S, \dashv\rangle)$
  - The m-state set R and the state transition function ϑ are computed starting from I_0

Algorithm

```
def construct_pilot():
  R' := {I0}
  do
    R := R'
    for each (m-state in R and symbol X in the pilot alphabet) do
      I' := closure(theta(I, X))
      if (I' is not empty set) then
        add_arc(theta, I -> I', X)
        if (I' not in R) then
          add_m_state(I', R')
        end
      end
    end
  while (R /= R')
```

Classification for m-states and pilot moves

- **Base set**: all the non-initial candidates of an m-state

- **Closure set**: all the initial candidates of an m-state
- **Kernel**: the projection on the first component of every candidate

$$I_{kernel} = \{q \in Q | \langle q, \pi \rangle \in I\}$$

  If two m-states have the same kernel, they are called kernel-equivalent.

A pilot m-state I has MTP (**multiple-transition property**) if it **includes two candidates** $\langle q, \pi \rangle$ **and** $\langle r, \rho \rangle$**,** $q \neq r$**, such that for some grammar symbol both transitions** $\delta(q, X), \delta(r, X)$ **are defined**.

The m-state I and transition $\vartheta(I, X)$ are said to be **convergent if** $\delta(q, X) = \delta(r, X)$.

A convergent transition has a **convergence conflict if** $\pi \cap \rho \neq \varnothing$.

ELR(1) condition

An EBNF grammar meets the ELR(1) condition if it satisfies the following the following requirements:

1. Every m-state I satisfies **both** the next clauses:
   - **no shift-reduce conflict**: for all candidates $\langle q, \pi \rangle \in I$ such that q is final and for all arcs $I \xrightarrow{a} I'$ it must hold $a \notin \pi$.
   - **no reduce-reduce conflict**: for all candidates $\langle q, \pi \rangle, \langle r, \rho \rangle \in I$ such that q, r are final it must hold $\pi \cap \rho = \varnothing$.
2. **No transition** of the pilot graph has a **convergence conflict**.

If the grammar is **purely BNF**, then the previous condition is referred to as **LR(1)** instead of ELR(1). **Convergence conflicts never occur in the BNF grammars**.

How the PDA works under the control of the pilot

1. The PDA **scans a string and executes a sequence of shift and reduction moves**.
2. The PDA **pushes groups of items and starts from those in the initial pilot m-state**.
3. Each **m-state** item becomes a **3-tuple called a stack candiate** by **adding** to it a **backward-directed pointer** that helps to **reconstruct the different analysis threads constructed in parallel**. The **set of stack candidates equivalent to an m-state is called a stack m-state (sms)**.
4. The PDA **decides whether to scan or reduce** basing on the look-ahead in the pilot
5. If the condition ELR(1) is satisfied (all parts), then the PDA is deterministic.

The **stack is** made as follows: $J_0 a_1 J_1 a_2 \dots a_k J_k$. The **current sms is the top of the stack and determines the next move**: either a **shift** tat scans the next token or a **reduction of a stack segment (handle)** to one of the final states in the current sms. This method leaves the **problem of finding the length of the stack handle**: that is why each stack candidate has a **pointer** (equivalent to an **integer** #i **indicating the index of the previous candidate** in the previous sms). The **pointer chain** is **followed backwards until** $\perp$ is encountered.

More formally, we can define an **algorithm for parsing**:

1. **Initialization** - The initial stack **contains the initial sms** equivalent to the initial m-state
2. **Shift move** - Let the top sms be J and I its relative m-state, let the current token be $a \in \Sigma$. Assume $\exists \vartheta(I, a) = I'$. A shift does the following:
   - **Push** a **onto the stack** and **gets the next token**
   - **Push on the stack** J' **the sms** equivalent to I'
3. **Reduction move (not initial)** - Let us assume that after inspecting the topmost sms the pilot makes a reduction. **From the top a chain starts which links each candidate to its predecessor. The chain ends when we encounter a candidate with a** $\perp$ **pointer**.

   Considering k the top and h as the end of chain, the reduction move proceeds as follows:
   - We grow the syntax tree by reducing $a_{h+1} a_{h+2} \dots a_k \rightarrow A$
   - We pop $J_k a_k J_{k-1} a_{k-1} \dots J_{h+1} a_{h+1}$
   - We execute a non-terminal shift move $\vartheta(J_h, A)$

4. **Initial state reduction move** - It differs from the previous case in the **chosen state, which is final and initial**. The parser grows the forest by reducing $\epsilon \to A$ and performs a non-terminal shift on the top of stack $\vartheta(J_k, A)$
5. **Non-terminal shift move** - The same as the aforementioned shift move, except that the symbol is a non-terminal. We do the same things except **we do not read the next input token**.
6. **Acceptance** - The parser **accepts and halts when the stack contains only** J_0, the **move is the non-terminal shift defined by** $\vartheta(J_0, S)$ **and the current token is** $\dashv$.

Complexity

When analyzing a string of length n, the **number of elements on the stack is at most** $n + 1$. To count the PDA moves we consider:

- n_t the number of terminal shifts
- n_n the number of non-terminal shifts
- n_r the number of reductions

It holds that $n_t = n$ **and that** $n_n = n_r$**. Thus the total number of PDA moves is** $n_t + n_n + n_r = n + 2n_r$. Furthermore:

- The **number of reductions with one or more terminal** (e.g $A \to aB$) is **at most** n
- The **number of null or copy reductions** is **linearly bounded** by n
- The **number of reductions without terminals** is **linearly bounded** by n

Thus both the **time** and **space** complexity are $\mathcal{O}(n)$.

Implementing the PDA with a vectored stack

In an implementation of the PDA with some **programming language**, we can always **mine the stack elements underneath the top one and directly look deep inside the stack**. Thus, the **third field of a stack item can be an integer that directly points back to the position of the stack element where the analysis thread begins**. This modification makes the **stack alphabet infinite** and implies that out automata is no longer a simple PDA.

Such modification is possible in every analyzer of practical interest and is not costly.

Top-down analysis

We can think of top-down analysis as a **set of syntactic procedures**, one **equivalent to one automaton of the EBNF machine net**, that **invoke each other (even recursively)**. A parser that operates in this way operates by **recursive descent**.

If we want such a parser to be **deterministic**, however, we need to **impose some further conditions on the machine net**.

By applying some modification, we can also **derive the ELL parser from the ELR one for the same grammar**.

ELL(1) condition

Recall the MTP property outlined previously. **We say that an m-state has the single-transition property (STP) if MTP does not hold**.

We can say that a machine net **meets the ELL(1) condition if all the following clauses are respected**:

1. There are **no left-recursive derivations**
2. The net **meets the ELR(1) condition**
3. The net **has STP**

Derivations of ELL(1) parsers

We can **start from an already constructed ELR(1) parser** and **apply the extra restrictions** of the ELL(1) condition to simplify the pilot.

Pilot compaction

Thanks to STP, we **can reduce the number of m-states downto the number of machine net states**.

Recall the relation kernel equivalency introduced before. We can say that **kernel-equivalency is an equivalence relation between various m-states**. It can be proven that **under STP**, **kernel-equivalent states can be coalesced into a single state** without affecting the power of the parser.

Candidate pointers removal

Thanks to STP, **convergent transitions are not possible anymore**. This means that **we can eliminate stack pointers from our stack since we need to carry out only one analysis thread**. This also means that **we can delete from each sms all candidates relative to unfollowed analysis threads**.

Stack contractions and predictive parser

As we modeled our parser before, with syntactic procedure, **we can restructure completely how our pilot uses the stack**:

- Since only **one thread is followed**, we need to **only store the sequences of machines invoked and the current state of the current machine** (the one at the top of stack) is in
- **Each machine before the current one** is in a **"suspended state"**. For each of these machines, **we need to store also the return state** from where the computation will resume after the current machine has finished.

From these considerations, we **adjust the compacted pilot graph to be isomorph to the machine net to obtain the control-flow graph** of the parser

1. Every **m-node** that contains **many candidates** is **split in as many nodes**
2. **Kernel equivalent nodes are coalesced** into a single node and the lookahead sets merged
3. We create new arcs, called **call arcs**, which **represent transfer of control from one machine to another**.

   Each call arc is **labeled with a set of terminals, the guide set**, which will determine the parser decision to transfer control to the called machine.

The **guide set** of a call arc $Gui(q_A \to 0_{A1})$ is **defined as follows**:

1. For every arc $q_a \overset{\gamma_1}{\to} 0_{A1}$ associated with non-terminal shift arc $q_a \overset{A1}{\to} r_A$, a terminal b is in the guide set γ_1 **iff one of the follwing is true**

$$
\begin{aligned}
& b \in Ini(L(0_{A1}) \\
& A_1 \text{ is nullable } \wedge b \in Ini(L(r_a)) \\
& A_1, L(r_a) \text{ both nullable } \wedge b \in \pi_{r_a} \\
& \exists (0_{A1} \overset{\gamma_1}{\to} 0_{A2}) \in \mathscr{F} \wedge b \in \gamma_2
\end{aligned}
$$

2. For every **terminal shift arc** $p \overset{a}{\to} q$ we set $Gui(\cdot) = \{a\}$
3. For every **dart that tags a final node** containing candidate $\langle f_A, \pi \rangle$ (f_A final) we set $Gui(\cdot) = \pi$).

The **essential property of guide sets** is the following:

1. **For every node** q of the graph of a grammar that satisfies the ELL(1) condition, **the guide-sets of any two arc originating from** q **are disjoint**.
2. If the **guide sets of control-flow graph are disjoint**, then the machine net **satisfies the ELL(1) condition**.

Construction of an ELL(1) by means of recursive procedures

In this construction **each machine becomes a procedure without parameters**. Then we can map pilot moves to actions as follows:

1. Call move: procedure call

2. Scan move: call `next` (reads and returns the next character)
3. Return move: returns from procedure

The **parser control graph, then becomes the control graph of the procedure**. We use the guides sets and the look-ahead sets (in this case also called prospect sets) to choose the next move. The analysis starts by calling the axiomatic procedure.

Direct construction of the PCFG (control-flow graph)

We do not need to first construct the ELR parser and then simplify it to ELL, we can **just build the PCFG directly from the machine net**.

1. **Prospect states**: we can reuse the same rules as the look-ahead sets of the ELR. Alternatively, we can use the following equations to compute the sets for each state, even non-final:
   - For an initial state:

$$\pi_{0_A} = \pi_{0_A} \bigcup_{q_i \xrightarrow{A} r_i} (Ini(L(r_i)) \cup \textbf{ if } Nullable(L(r_i)) \textbf{ then } \pi_{q_i} \textbf{ else } \varnothing)$$

   - For any other state:

$$\pi_q = \bigcup_{q_i \xrightarrow{X_i} r_i} \pi_{p_i}$$

2. **Guide sets**: we use the prospect sets previously calculated:
   - For a call arc (image because I do not want to write latex for this abomination):

$$Gui\,(q_A \dashrightarrow 0_{A_1}) := \bigcup \begin{cases} Ini\left(L\,(A_1)\right) \\ \hline \textbf{if } Nullable\,(A_1) \textbf{ then } Ini\left(L\,(r_A)\right) \\ \textbf{else } \emptyset \textbf{ endif} \\ \hline \textbf{if } Nullable\,(A_1) \wedge Nullable\left(L\,(r_A)\right) \textbf{ then } \pi_{r_A} \\ \textbf{else } \emptyset \textbf{ endif} \\ \hline \bigcup_{0_{A_1} \dashrightarrow 0_{B_i}} Gui\,(0_{A_1} \dashrightarrow 0_{B_i}) \end{cases}$$

Figure 1: Guide set equation for call arcs

   - For final darts and terminal shifts we have respectively:

$$Gui(f_A \rightarrow) = \pi_{f_A}$$
$$Gui(q_A \xrightarrow{a} r_A) = \{a\}$$

   - Initially all guide sets are empty

Increasing look-ahead

If the ELL(1) condition is not verified, we could try to modify the grammar and make it compliant. However this approach is very costly and not always possible. An alternative approach is to **use a longer look-ahead**, like ELL(2).

With $k > 1$ things are an extensions of $k = 1$: **the analyzer looks at k consecutive characters (or tokens) in the input; if the guide set of length k on alternate moves are disjoint, then the ELL(k) condition is satisfied and analysis is possible**.

Static analysis

Compilation is done in different stages:

1. The **source** is **translated into an intermediate form** more similar to the machine language
2. On the intermediate form we can **verify**, **optimize** and eventually **change the order of the instruction flow** to improve execution efficiency.

It is convenient to **model the control flow graph** of the program as an **automaton** and to process such an automaton to carry out analysis since we can efficiently process FSAs. Note that **the automaton defines a single program** and not the entire source programming language.

Static flow analysis consists of studying some properties of the program by means of the methods and techniques of automata theory, formal logic and statistics.

Control flow graph

Let us consider **only the simplest instructions**: scalar variable and constant, assignment, simple unary arithmetic, logic or relation operation. Let us consider only **intraprocedural** analysis.

In the **program control graph** every **node** represents a **program instruction**. If **instruction p is followed by q, then the graph has an arc directed from p to q**. The **first instruction** is the **input node**, while an **instruction without successors** is the **final one**. Unconditional instructions have one successor, where as **conditional ones have multiple**. An instruction with **two or more predecessors** is a **confluence node** (also called merge/join).

The control graph is not a complete and faithful representation of the program:

- The **logical value of a condition is not represented**
- The **assignment operation is abstracted**:
  - Assigning or reading creates the variables
  - Referencing a variable uses that variable

The FSA represented by the control flow graph has an **instruction set I as terminal alphabet** and each **instruction** is defined as **a triple with node name and sets *def* and *use***. The **formal language L recognized by said FSA contains the strings over the alphabet I that label the paths from the initial state to a final state**. Each **path** represents a **sequence of program instructions the machine can execute when the program is run**. Language L **is local**, i.e a subfamily of REG.

The graph **may contain sequences of instructions that are not executable** as it does not model the clauses of the conditional instructions. In general **it is undecidable whether a path in the control flow graph is executable** or not, as the **Turing halting problem** is reducible to such a problem.

Hypothesis: for static analysis to be meaningful, the **control flow graph has to be in reduced form**.

Liveness

Liveness interval of a variable: a variable is **live at a point p**, if the program control flow graph **has a path from point p to another point q and both conditions below hold**:

- the path $p \to q$ does **not pass through any instructions $r \neq q$ that defines variable a** ($a \in def(r)$)
- instruction q **uses variable** a, ($a \in use(q)$)

Informally: a variable is live if some instruction that later can be executed starting from p uses the value the variables happens to have at point p.

Let I be the set of all instructions, $D(a), U(a) \subseteq I$ be the sets o instructions that define and use variable a. **A variable a is said to be live if the following condition holds** for the language accepted by the automaton:

- **Liveness condition**: a variable a is live at the output of p if **in** $L(A)$ **generated by the control graph automaton** A **there is** $x = upvqw$ such that:

$$\begin{aligned} & u, in I^\star \wedge \\ & \quad p \in I \wedge \\ & \quad v \in (I \setminus D(a))^\star \wedge \\ & \quad q \in U(a) \end{aligned}$$

  The set of **all the strings** x **that meet this condition** is a **regular language** $L_p \subseteq L$. To **check if a variable** a **is live** at the output of p, we check if the **language** L_p **is empty**.

$$\begin{aligned} L_p &= L(A) \cap R_p \\ R_p &= I^\star p (I \setminus D(a))^\star U(a) I^\star \end{aligned}$$

To **check the emptyness** of L_p **we can build a recognizer** and check if there are paths that connect the input node to some final node. However, this method is **slow**. A **new method** can determine **all live variables simultaneously**.

- For every **final node** p:

$$live_{out}(p) = \emptyset$$

- For every other **node** p:

$$\begin{aligned} live_{in}(p) &= use(p) \cup (live_{out}(p) \setminus def(p)) \\ live_{out}(p) &= \bigcup_{\forall q \in succ(p)} live_i n(q) \end{aligned}$$

  Where $succ(p)$ is the set of all nodes that directly follow p.

For a graph with a number $n > 1$ of nodes, we have a set of $2n$ equations that contains $2n$ unknowns. We **solve the equation set iteratively by initially assigning the empty set to each unknown** as follows: $live_{in}(p) = \emptyset$ and $live_{out}(p) = \emptyset$. Then **in the current solution** i **we replace the equation values and number the new solution as** $i + 1$. We go on **until convergence**.

The computation **converges after a finite number of steps** because:

1. Every set has a cardinality upper bounded by the total number of variable in the program
2. Every iteration step either increases the cardinality of the above sets or at worst leaves them unchanged
3. When the solution does not change any more, it terminates

Useless instructions: a **definition instruction is useless** if the variable is **not live on at least one outgoing arc of the instruction**. To identify useless instructions we **first search an instruction** p **that defines** a **then we check whether** a **belongs to** $out(p)$.

Reaching definition

Now we analyze whether a **variable definition reaches an instruction of the program**.

Reaching definition: the **definition of a variable** a **in instruction** q **reaches the input of an instruction (**p **and** q **may coincide) if there is a path from** q **to** p **the inner nodes of which do not define the variable** a **again**.

Equivalent definition:

$$u, w \in I^\star \wedge q \in D(a) \wedge v \in (I \setminus D(a))^\star \wedge p \in I$$

$$sup\,(p) = \left\{ a_q \;\middle|\; \begin{array}{l} q \in I \;\wedge\; q \neq p \;\wedge \\ a \in def(q) \;\wedge\; a \in def(p) \end{array} \right\} \quad \text{if } def(p) \neq \emptyset$$

$$sup\,(p) = \emptyset \quad \text{if } def(p) = \emptyset$$

Figure 2: Sup

The condition can be **reformulated as flow equations**. If a **node** *p* **defines a variable** *a*, then **every other definition** a_q **of the same variable** *a* in some node *q* is said to be **suppressed by** *p*. Assume *q* **is any node**, not necessarily one that reaches *p*, the set *sup*(*p*) **of the definitions suppressed by** *p* **is expressed** as seen in:

$def(p)$ may contain two or more variables names.

The **flow equations** are the following:

for the initial node *1*

$$in\,(1) = \emptyset$$

for every other node *p*

$$out\,(p) = def(p) \;\cup\; \left(in\,(p) \setminus sup\,(p) \right)$$

$$in\,(p) = \bigcup_{\forall\, q \in pred\,(p)} out\,(q)$$

Figure 3: Flow equation reaching

1. The first equation assumes there are not any variables passed as input parameters. Otherwise the set *in*(1) may also contain some definitions external to the subprogram.
2. The second equation puts the definitions of *p* into the output of *p*, and puts therein any other definition that reaches the input of *p* and is not suppressed by *p* itself
3. The third equation collects all the definitions that reach the output of any node that immediately precedes *p*, and puts all of them into the input of *p*

This is a **forward-propagation** computation scheme! We solve the above system of equations iteratively, **similarly to the liveness problem**.

Syntax transduction

Syntax transduction (or translation) is an **abstract definition of the correspondence between two formal languages**. The translation is **based on local transformations of the source text**: it replaces each character with another one or with an entire string and depends on a table that specifies how to transliterate the source alphabet into the destination one.

There are **two types** of transduction models:

1. **Regular**: uses a **finite transducer**
2. **Context-free (syntactic)**: uses a **free grammar** to define the source and destination languages

Transduction relation and function

Given a source and a destination alphabet Σ and Δ, a **transduction is a correspondence of strings that is modeled as a mathematical binary relation** between the universal languages $\Sigma^\star$ and $\Delta^\star$.

The **transduction relation ρ is a set of pairs** (x, y) with $x \in \Sigma^\star$ and $y \in \Delta^\star$ and the languages L_1, L_2 **are the projections of ρ over the first and second component**.

There are the following seven cases, depending on the **function** type:

1. Total: every string as at least one translation
2. Partial: there exists a string that is not translatable
3. One-valued: very source string has an image of at most one destination string
4. Multi-valued: a few source strings may have an image of two or more destination strings
5. Injective: any two different strings have different translations
6. Surjective: any destination string is the image of one or more source strings
7. Bijective: there is one-to-one correspondence between source and destination

A transduction is bijective if and only if it is both injective and surjective. Such translation is invertible and its inverse is bijective as well.

Transliteration

The **simplest** translation: **every source character is translated into a corresponding destination character, or more generally into a string**. It is also called homomorphism.

Transliteration is **surely a one-valued transduction**, but as said before **in general its inverse is partial and may be multi-valued**: if a **transliteration can cancel a source character** and replace it by the empty string, then **its inverse transduction is surely multi-valued** and actually infinitely-valued.

Regular transduction

In order to define a transduction relation one can **exploit regexps**: in this case the **regular operators** union, concatenation and star **are applied in parallel to pairs of strings rather than to individual strings**.

A **regular or rational transduction expression r is a regexp** that contains union, concatenation and star (cross) operators, **the terms of which are pairs** (u, v) **of strings (possibly empty) over the source alphabets**.

Regular transductions can be **recognized by a two-tape automaton**. We consider the **pairs in our transduction as terminal symbols**. When the labels of the arcs of a **two-tape automaton are projected over the first component one obtains a pure recognizer**: this is called the **underlying input automaton** and it is the **recognizer for the pure source language**.

Like with FSA, we can determine the **determinism for double-taped automatons**. It must be a **deterministic machine** that has a finite amount of states, a source alphabet Σ, an initial state q_0, a subset F of final states and **three one-valued functions**:

1. The **transition function** δ, which computes the next state
2. The **output function** ν, which computes the string to write in each move
3. The **final function** ϕ, which computes the last suffix to concatenate to the output string when the transducer reaches a final state.

Purely syntactic transduction

See **book**.

Attribute grammars

See **book**.